ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ

СЕТЕВЫЕ ТЕХНОЛОГИИ · НЕДЕЛЯ 1 · ЛОНГРИД

Стек протоколов и прикладной уровень

Как приложение проходит путь от имени и DNS до HTTP-ответа — и как превратить сетевой симптом в проверяемое наблюдение.

Материалы курса · запуск 2026

Содержание

01 Зачем разработчику понимать сеть

02 Результат обучения

03 1. Начнём со знакомого: HTTP

04 2. Что именно задаёт протокол

05 3. HTTP описывает не весь путь

06 4. DNS: от имени к адресу и другим данным

07 5. Собираем полный путь запроса

08 6. Диагностика начинается с фактов

09 7. Как сформулировать полезное обращение

10 8. Самопроверка

11 Источники и дальнейшее чтение

Зачем разработчику понимать сеть

Разработчики постоянно слышат одну и ту же фразу:

> Сайт не работает.

За ней могут скрываться принципиально разные наблюдаемые результаты:

- имя не удалось преобразовать в адрес;
- соединение с адресом отклонено;
- операция завершилась по тайм-ауту;
- HTTP endpoint вернул `503 Service Unavailable`;
- HTTP-ответ получен, но приложение не смогло интерпретировать его содержимое.

Во всех случаях пользователь может описать ситуацию одинаково. Для разработчика этого описания недостаточно. Наша задача — научиться раскладывать сетевое взаимодействие на этапы и понимать, какой факт подтверждён на каждом этапе.

Курс не готовит сетевых инженеров и не требует умения настраивать маршрутизаторы. Его цель — дать разработчику рабочую модель сети: понимать, как она влияет на приложение, замечать типичные грабли, собирать первичные наблюдения и предметно обращаться к SRE или сетевым инженерам.

Результат обучения

После связки «лекция + семинар» студент должен уметь:

1. По URL восстановить путь от разрешения имени до обработки HTTP-ответа.
2. Объяснить, какую ответственность несут прикладной, транспортный, сетевой и канальный уровни.
3. Прочитать базовый HTTP/1.1-запрос и ответ.
4. Различить ошибку разрешения имени, отказ при установлении соединения, тайм-аут и HTTP-ответ с кодом 4xx или 5xx.
5. Отделить наблюдаемый факт от гипотезы и предложить следующую проверку, которая сузит область поиска.

Цель первой недели — не выучить все прикладные протоколы и не научиться настраивать DNS-сервер. Мы строим первую рабочую модель, к которой будем добавлять детали на следующих неделях.

1. Начнём со знакомого: HTTP

Возьмём URL, похожий на тот, который ежедневно обрабатывает браузер или HTTP-клиент:

```
https://example.com/catalog?page=2#reviews
```

Его части задают:

- `https` — scheme; для неё по умолчанию выбирается порт 443;
- `example.com` — host, то есть имя узла;
- `/catalog` — path;
- `page=2` — query component;
- `reviews` — fragment.

Fragment относится к состоянию клиента и обычно не передаётся серверу в HTTP-запросе. URL задаёт логическую цель обращения, но пока не объясняет, как данные попадут на другой компьютер.

На самом верхнем уровне HTTP выглядит как простой обмен:

```
клиент ── HTTP request ──▶ сервер
клиент ◀── HTTP response ── сервер
```

Клиентом может быть браузер, мобильное приложение или другой backend. Сервером может быть origin application, reverse proxy, gateway или иной HTTP endpoint. Важны роли участников, а не названия конкретных программ.

HTTP описывает прикладное взаимодействие

HTTP — stateless application-level protocol с моделью request/response. Он описывает работу с ресурсами, методы, поля и status codes. HTTP не объясняет, как разрешается DNS-имя, как IP-пакеты идут через сеть и как браузер отрисовывает страницу.

HTTPS — это HTTP, защищённый TLS. Семантика HTTP при этом не превращается в другой прикладной протокол. TLS подробно рассматривается на следующей неделе.

Ресурс и представление

Ресурс — логическая цель запроса. Полученные байты являются представлением состояния ресурса, а не самим ресурсом. Один ресурс может иметь разные представления, например JSON и HTML.

Это различие помогает понимать поля `Content-Type`, `Content-Encoding`, content negotiation и кэширование. На первой неделе достаточно помнить: HTTP передаёт сообщения о ресурсах и их представлениях.

Как читать HTTP/1.1-запрос

```
GET /catalog?page=2 HTTP/1.1
Host: example.com
Accept: application/json
```

Запрос состоит из:

1. Request line: method, request target и HTTP version.
2. Field lines: метаданные сообщения.
3. Пустой строки, завершающей секцию полей.
4. Необязательного content.

Пустая строка — часть формата. В текстовом представлении её легко не заметить, но без неё граница между полями и содержимым не определена.

`Host` сообщает, к какому имени и порту относится запрос. Это позволяет нескольким виртуальным хостам обслуживаться на одном адресе.

Методы — не просто CRUD

Method выражает семантику запроса:

- `GET` запрашивает текущее представление ресурса;
- `HEAD` ведёт себя как `GET`, но сервер не включает content в ответ;
- `POST` передаёт content для обработки в соответствии с семантикой ресурса;
- `PUT` просит создать или заменить состояние целевого ресурса переданным представлением;
- `DELETE` просит удалить связь между URI и текущей функциональностью ресурса;
- `OPTIONS` запрашивает сведения о доступных способах взаимодействия.

Связывать методы только с CRUD удобно как первое приближение, но этого недостаточно для точного проектирования API. Свойства safe и idempotent будут важны при обсуждении повторов запросов, однако не являются обязательной целью первой недели.

Как читать HTTP/1.1-ответ

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 17

{"items":[1,2,3]}
```

Ответ состоит из:

1. Status line: HTTP version, status code и необязательная reason phrase.
2. Field lines.
3. Пустой строки.
4. Необязательного content.

Status code — не «код ошибки». Класс 2xx тоже содержит status codes. Первая цифра задаёт общий класс результата:

- 1xx — промежуточная информация;
- 2xx — запрос успешно обработан;
- 3xx — для завершения требуется дополнительное действие;
- 4xx — сервер связывает результат с запросом клиента;
- 5xx — сервер не смог выполнить корректно принятый запрос.

HTTP-ответ и отсутствие ответа — разные факты

Если клиент получил `503 Service Unavailable`, некоторый HTTP endpoint или intermediary смог сформировать HTTP-ответ. Это не доказывает доступность всех upstream-компонентов и не означает успех бизнес-операции, но граница диагностики уже отличается от тайм-аута до получения ответа.

```
503 Service Unavailable
```

Подтверждено: получен и разобран HTTP-ответ.

```
Operation timed out
```

Подтверждено: операция не завершилась в заданное время. Неизвестно, был ли HTTP-ответ сформирован где-либо по пути.

Stateless не означает «состояния нет»

HTTP позволяет рассматривать каждый запрос самостоятельно: сервер не обязан восстанавливать смысл текущего запроса из скрытого контекста предыдущего обмена. Но приложение может хранить состояние:

- в серверной сессии;
- во внешнем хранилище;
- в cookie или token;
- в client-side storage;
- непосредственно в URL или content запроса.

У HTTP-соединения также есть собственное транспортное состояние. Нельзя переносить слово stateless на всю систему целиком.

HTTP/1.1, HTTP/2 и HTTP/3

У версий HTTP общая прикладная семантика: methods, status codes, fields и понятие ресурса. Различается представление и способ передачи сообщений:

- HTTP/1.1 имеет удобное для чтения текстовое представление;
- HTTP/2 использует binary framing и multiplexing;
- HTTP/3 переносит HTTP-семантику поверх QUIC.

Поэтому HTTP/2 — не другой прикладной протокол, а другая версия той же протокольной семантики. Детали относятся к следующим неделям.

2. Что именно задаёт протокол

HTTP показывает, что протокол — это не просто формат байтов. Протокол задаёт:

- роли участников;
- формат сообщений;
- смысл полей и действий;
- допустимую последовательность обмена;
- правила поведения при известных условиях и ошибках.

Если две реализации одинаково разбирают байты, но по-разному понимают их смысл, совместимого протокола не получилось.

Прикладной уровень не заканчивается HTTP

HTTP удобен как первый пример, потому что разработчики постоянно встречают его в браузерах, API и backend-взаимодействиях. Но разные прикладные задачи требуют разных протоколов и разных моделей диалога:

Задача	Протокол	Упрощённая модель взаимодействия
Получить представление ресурса	HTTP	request → response
Найти типизированные данные по имени	DNS	query → answer
Передать почтовое сообщение	SMTP	store → forward
Работать с почтовым ящиком на сервере	IMAP	удалённое состояние и синхронизация
Получить сохранённые сообщения	POP3	простой доступ к сообщениям
Передать файлы	FTP	отдельные управление и данные

Один пользовательский сценарий может собираться сразу из нескольких протоколов. Например, упрощённый путь электронного письма выглядит так:

```
почтовый клиент отправителя
    └─ SMTP submission → сервер отправителя
                              └─ DNS MX → выбор сервера получателя
                              └─ SMTP relay → сервер получателя
                                                  ├─ IMAP → работа с ящиком
                                                  └─ POP3 → получение сообщений
```

SMTP используется для отправки сообщения и его передачи между почтовыми системами. MX-записи DNS позволяют определить серверы, принимающие почту для домена. IMAP предоставляет модель удалённой работы с почтовым ящиком и синхронизации состояния. POP3 предлагает более простую модель получения сохранённых сообщений.

Однако успешная доставка письма ещё не объясняет, как внутри представлены русский текст, HTML, изображения и вложения. Передача сообщения и представление его содержимого — разные задачи. К этой границе мы вернёмся на второй неделе при обсуждении MIME, multipart, кодировок и сериализации.

RFC, IETF и актуальность документа

RFC расшифровывается как Request for Comments. Серия началась в 1969 году, когда Steve Crocker опубликовал RFC 1 `Host Software` как рабочую заметку проекта ARPANET и приглашение обсуждать инженерные решения. Со временем RFC стали архивным каналом публикации спецификаций, процедур, исследований и инженерного опыта об Интернете.

Многие современные интернет-стандарты развиваются в IETF и публикуются как RFC, но RFC шире стандартов IETF. Сам номер RFC не означает, что документ является действующим стандартом.

Документы имеют разные категории и статусы; более новый RFC может обновлять или заменять старый. Опубликованный текст RFC не переписывают: изменения оформляют новым документом и связями `Updates` или `Obsoletes`.

Перед использованием RFC нужно проверить:

- status и stream документа;
- категорию документа;
- связи `Updates` и `Obsoletes`;
- наличие errata;
- актуальное состояние на странице RFC Editor.

Например, общая семантика HTTP сейчас описана в RFC 9110, а формат сообщений HTTP/1.1 — в RFC 9112. RFC 2616 полезен исторически, но больше не является актуальной спецификацией HTTP/1.1.

3. HTTP описывает не весь путь

Мы научились читать HTTP-сообщение, но пока не ответили на вопрос: как его байты попали на другой компьютер? HTTP описывает смысл прикладного обмена, но не решает самостоятельно задачи адресации, доставки между сетями и физической передачи.

Уровни разделяют ответственность

Для разработчика достаточно следующей причинной модели:

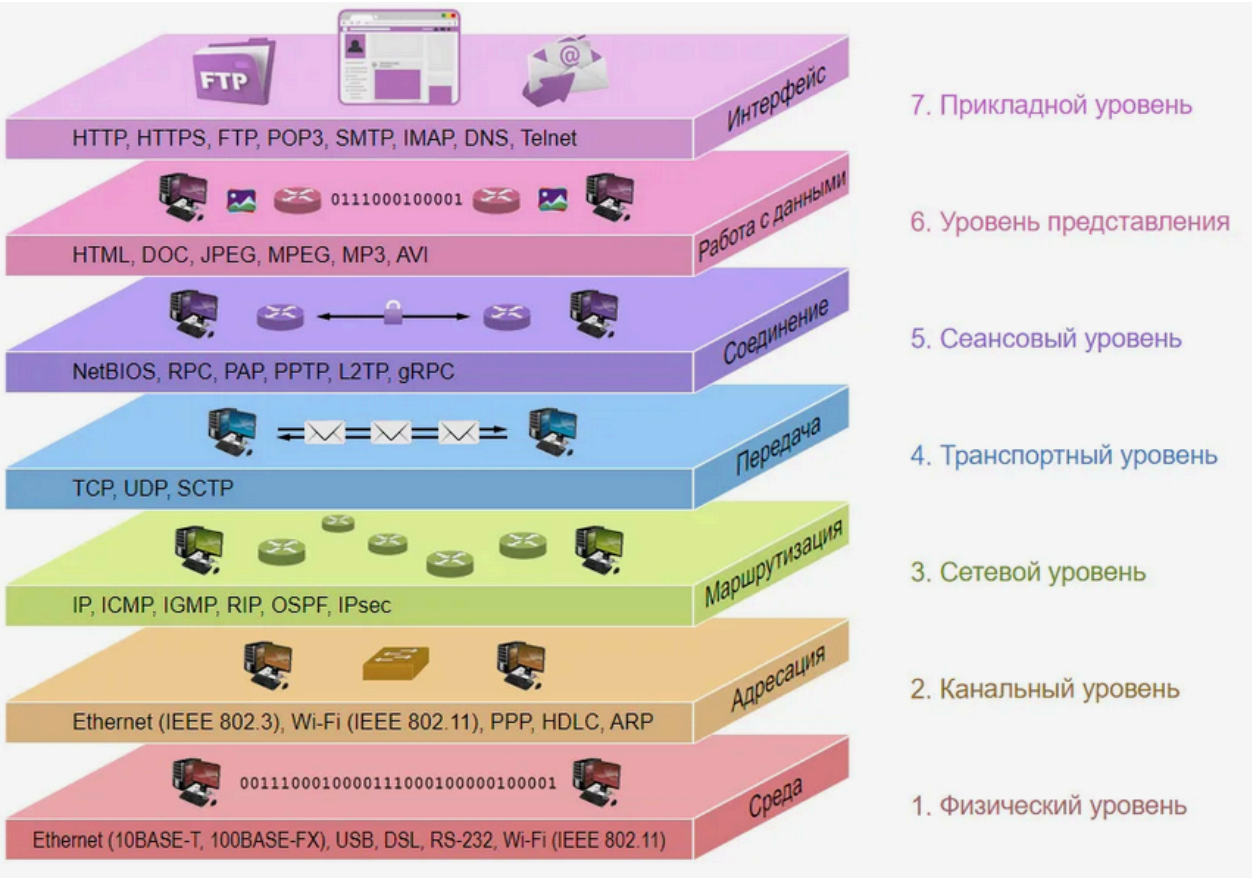
Уровень	Главный вопрос	Пример единицы данных
Прикладной	Что стороны хотят сообщить друг другу?	HTTP message, DNS message
Транспортный	Как связать процессы на конечных узлах?	TCP stream, UDP datagram
Сетевой	Как доставить данные между сетями?	IP packet
Канальный	Как доставить данные в локальном сегменте?	Ethernet frame
Физический	Как перенести сигнал по среде?	электрический, оптический или радиосигнал

Уровни — это модель зон ответственности, а не физический запрет устройству смотреть глубже. Коммутатор обычно принимает решения по информации L2, маршрутизатор — по L3, но firewall, load balancer или proxy может анализировать несколько уровней.

OSI и практический стек TCP/IP

В модели OSI семь уровней. В практическом стеке TCP/IP функции OSI L5–L7 часто не выделены в отдельные протоколы и рассматриваются вместе как application layer. Поэтому вопрос «к какому единственному уровню относится TLS?» не всегда имеет полезный однозначный ответ. Модель нужна, чтобы разделять ответственность, а не чтобы любой механизм обязательно поместился в одну из семи коробок.

Ниже приведена схема из материалов 2025 года. Она помогает увидеть идею уровней, но отдельные подписи на ней не следует воспринимать как строгую классификацию современных протоколов.



Один из вариантов визуализации модели OSI

Инкапсуляция

Когда приложение передаёт HTTP-сообщение, нижние уровни добавляют собственную служебную информацию:

```
Ethernet frame
└── IP packet
    └── TCP data
        └── HTTP message
```

На принимающей стороне данные проходят обратный путь. Это не означает, что каждый промежуточный узел разбирает сообщение до HTTP. Большинство устройств обрабатывает только информацию, которая нужна ему для выполнения своей роли.

Приложение знает из URL имя `example.com`, но IP-уровню нужен адрес назначения. Это создаёт следующий этап нашего пути.

4. DNS: от имени к адресу и другим данным

DNS — Domain Name System, распределённая иерархическая система имён. Преобразование имени в IP-адрес — только один из её сценариев.

Для обращения к `app.course.test` клиенту обычно нужны записи `A` и/или `AAAA`:

```
app.course.test.  A     192.0.2.10
app.course.test.  AAAA  2001:db8::10
```

Одному имени могут соответствовать несколько адресов. Адрес может измениться. Полученный ответ может зависеть от местоположения клиента, политики сервиса и содержимого кэша.

Домен `.test` и адресные диапазоны `192.0.2.0/24` и `2001:db8::/32` используются здесь как безопасные примеры, а не как реальные публичные сервисы.

Иерархия имён

Полное доменное имя читается справа налево:

```
app.course.test.
│   │      │   └─ root
│   │      └──── top-level domain
│   └─────────── domain
└─────────────── label внутри domain
```

Завершающая точка обозначает корень DNS. В пользовательском вводе её обычно опускают.

Ответственность за части дерева делегируется. Root servers не хранят адрес каждого сайта: они помогают найти серверы, ответственные за следующий участок иерархии.

DNS хранит типизированные записи

В лекции используются шесть распространённых типов:

- `A` — IPv4 address;
- `AAAA` — IPv6 address;
- `CNAME` — canonical name alias;
- `NS` — authoritative name servers зоны;

- `MX` — серверы, принимающие почту для домена; эти записи используются при доставке почты по SMTP;
- `TXT` — текстовые данные, используемые различными механизмами.

Позже встретятся `SOA`, `PTR`, `SRV`, `HTTPS` и другие записи. Их не нужно запоминать списком: тип выбирается задачей, которую решает потребитель данных. Главная модель — DNS является распределённой базой типизированных записей, а не только таблицей «имя → IP».

Resolver — не один сервер

Слово resolver может обозначать разные роли:

- stub resolver принимает запрос от приложения или библиотеки ОС;
- recursive resolver ищет итоговый ответ для клиента и использует кэш;
- authoritative server отвечает за данные своей зоны.

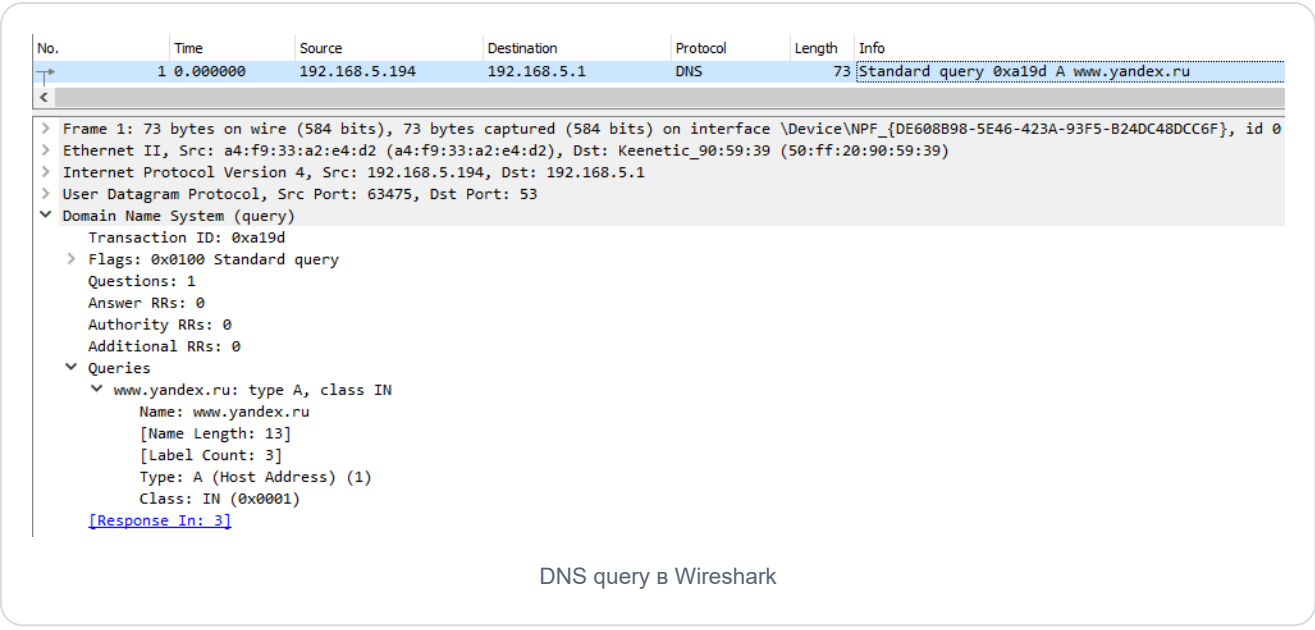
Упрощённая цепочка:

```
application → OS API / stub → recursive resolver
                                    ├─ cache hit → готовый ответ
                                    └─ cache miss → root → TLD → authoritative
```

На Linux приложение часто вызывает библиотечную функцию `getaddrinfo()`. Это не системный вызов ядра и не DNS-пакет. Дальнейший путь зависит от NSS, локального resolver service, контейнерной среды, настроек ОС и конфигурации. Нельзя универсально утверждать, что библиотека сама прочитает `/etc/resolv.conf` и немедленно отправит UDP-пакет.

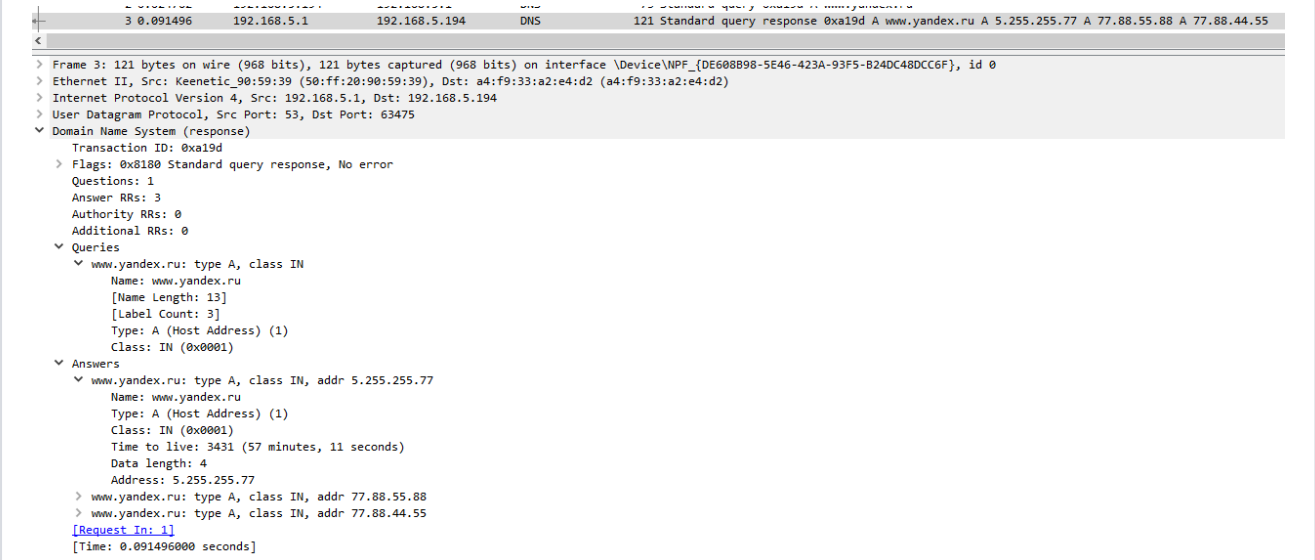
DNS-запрос и ответ можно наблюдать

На следующем снимке Wireshark показан DNS-запрос типа `A`. В дереве разбора одновременно видны Ethernet, IPv4, UDP и DNS: одно прикладное сообщение инкапсулировано в единицы данных нижних уровней.



DNS query в Wireshark

В ответе видны несколько `A` records и TTL. Один запрос имени не обязан возвращать один адрес.



DNS response в Wireshark

Эти снимки относятся к материалам 2025 года и используют реальный домен. Для упражнений 2026 года следует применять специально подготовленную среду или зарезервированные домены.

Кэш и TTL

TTL — Time to Live. В DNS он задаёт, сколько секунд закэшированный ответ можно использовать без нового запроса к источнику данных. Кэш уменьшает задержку и нагрузку, но означает, что изменение authoritative данных не обязано мгновенно проявиться у всех клиентов.

Например:

```
t = 0      resolver получил A 192.0.2.10 с TTL = 300
t = 120    authoritative DNS уже возвращает A 192.0.2.20
t < 300    кэш всё ещё может возвращать 192.0.2.10
t > 300    после истечения TTL следующий запрос обновляет запись
```

Это не обещание точного времени распространения изменения. Существует несколько уровней кэша, а реализации могут дополнительно ограничивать или переопределять срок хранения.

В диагностике нужно фиксировать не только имя и полученный адрес, но также место и время наблюдения.

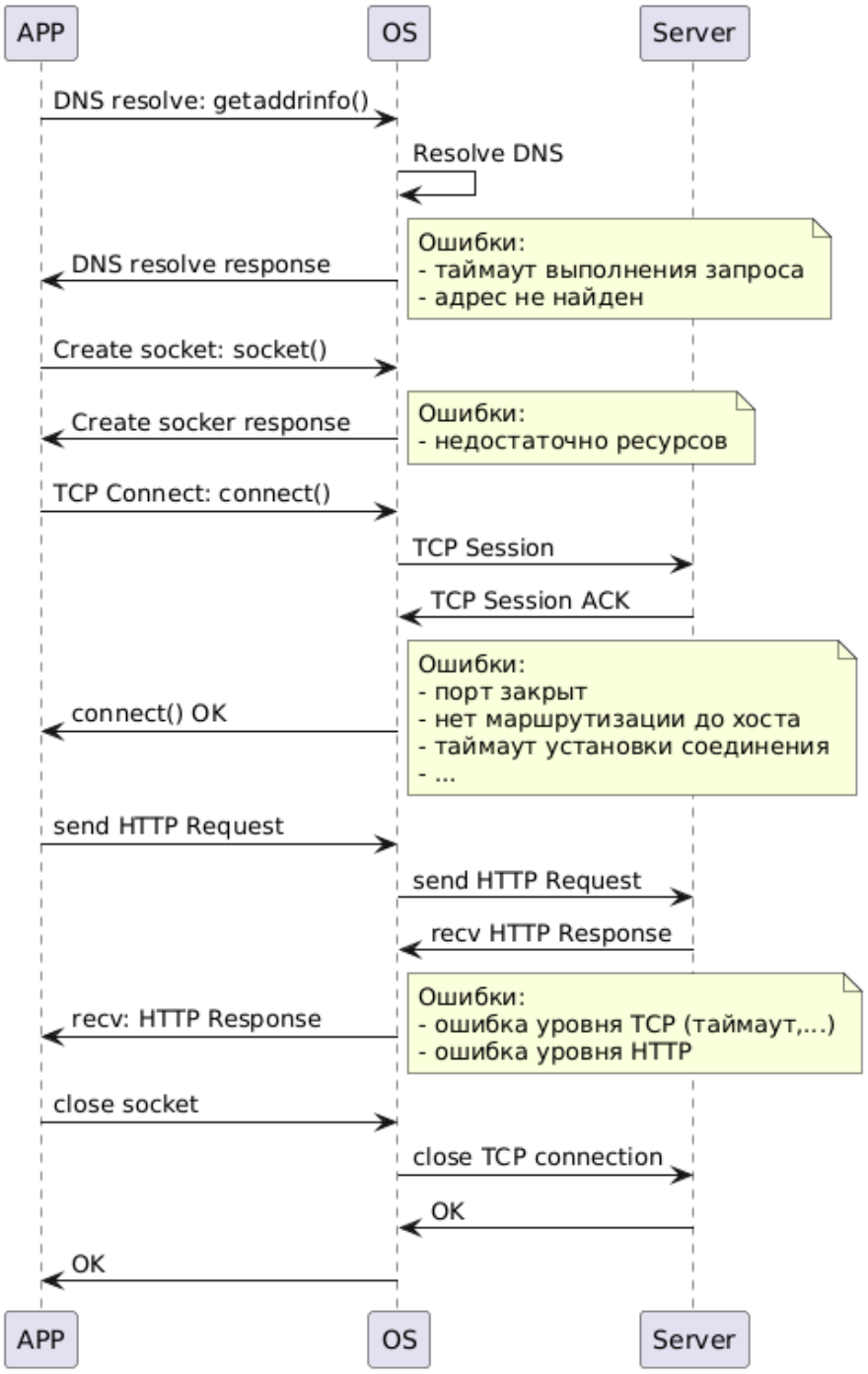
DNS не равен UDP/53

Классический DNS часто использует UDP-порт 53, но TCP/53 является штатной частью протокола и обязателен для современных реализаций. Существуют защищённые способы передачи DNS, включая DNS over TLS и DNS over HTTPS.

DNSSEC решает другую задачу: позволяет проверять подлинность и целостность DNS-данных, а не заменяет транспорт.

5. Собираем полный путь запроса

Упрощённая диаграмма из материалов 2025 года показывает взаимодействие приложения, ОС и сервера:



Упрощённый путь HTTP-запроса

Диаграмма полезна как карта этапов, но требует двух оговорок:

1. Реализация разрешения имени зависит от ОС и окружения.
2. Между приложением и origin server могут находиться proxy, load balancer, CDN и другие intermediaries.

Для первого приближения используем цепочку:

```
1. Клиент разбирает URL и извлекает scheme, host, port и path.
2. Resolver возвращает один или несколько адресов.
3. Клиент выбирает адрес и пытается установить transport connection.
4. Клиент формирует и отправляет HTTP request.
5. Клиент получает и разбирает HTTP response.
6. Приложение интерпретирует результат.
```

Если используется HTTPS, между connect и HTTP-обменом появляется TLS handshake. Его мы добавим в модель на второй неделе. Механика TCP-соединения, повторные передачи и влияние перегрузки будут разобраны на транспортном уровне.

Наблюдаемый результат каждого этапа

Этап	Возможные результаты
Разбор URL	URL принят или отвергнут клиентом
Resolve	адрес, `NXDOMAIN`, временная ошибка, тайм-аут
Connect	соединение установлено, отказ, тайм-аут
Отправка запроса	данные переданы транспортному уровню или произошла ошибка записи
Получение ответа	HTTP-ответ, неполный обмен, закрытие соединения, тайм-аут
Интерпретация	ожидаемое или неожиданное содержимое; успех или ошибка бизнес-операции

Такая таблица важнее списка возможных виновников. Она помогает сначала определить границу, до которой взаимодействие точно дошло.

6. Диагностика начинается с фактов

Рассмотрим сообщение клиента:

```
Operation timed out after 5000 milliseconds
```

Из него следует факт: клиент не завершил некоторую операцию за пять секунд. Можно предположить перегрузку сервера, потерю пакетов, ошибку маршрутизации, блокировку firewall или слишком маленький тайм-аут самого клиента. Но ни одна из этих причин ещё не доказана.

Полезная диагностическая формула:

```
наблюдение → этап → гипотезы → следующая проверка
```

Например:

```
Наблюдение: соединение с 192.0.2.10:8080 отклонено немедленно.
Этап: connect.
Гипотезы: на адресе и порту нет listener; выбран неверный адрес или порт;
          отказ сформировал промежуточный узел.
Следующая проверка: подтвердить адрес, порт, источник отказа и состояние listener.
```

Фраза «это сеть» не является наблюдением. Она слишком широка и не помогает выбрать следующую проверку.

`Could not resolve host`

Подтверждено: приложение не получило пригодный для дальнейшего соединения результат разрешения имени.

Не подтверждено: что authoritative DNS недоступен, что запись отсутствует или что «Интернет не работает». Ответ мог зависеть от локальной конфигурации, кэша, search suffix, выбранного resolver или ошибки в имени.

Следующая проверка: зафиксировать точное имя, тип запроса, используемый resolver и фактический DNS-результат.

`Connection refused`

Подтверждено: попытка соединения получила явный отказ.

Не подтверждено: что приложение на сервере «сломано». Возможны неверный адрес или порт, отсутствие listener, политика промежуточного узла и другие причины.

Следующая проверка: сверить адрес и порт, определить источник отказа, затем проверить состояние listener и путь до него.

`Operation timed out`

Подтверждено: ожидаемый результат не получен в установленный срок.

Не подтверждено: что причиной является firewall. Тайм-аут — наблюдаемый результат, а firewall — только одна из гипотез.

Следующая проверка: определить, какая именно операция ожидалась — resolve, connect, write или response — и собрать наблюдение на этой границе.

`HTTP/1.1 503 Service Unavailable`

Подтверждено: HTTP endpoint или intermediary сформировал ответ, а клиент получил и разобрал status line с кодом 503.

Не подтверждено: что origin application недоступно. Ответ мог сформировать origin, reverse proxy, load balancer или gateway.

Следующая проверка: сохранить поля ответа, определить ответивший компонент и сопоставить запрос с серверными наблюдениями.

`200 OK`, но пользователь видит ошибку

Подтверждено: HTTP-обмен завершился status code 200.

Не подтверждено: успех бизнес-операции. Content может содержать прикладную ошибку, устаревшие данные или неожиданный формат.

Следующая проверка: проверить содержимое ответа и контракт приложения.

7. Как сформулировать полезное обращение

Плохое обращение:

> Сеть не работает. Посмотрите, пожалуйста.

Полезное обращение:

> 10 августа в 14:32 MSK приложение `checkout` из окружения `stage` разрешило `api.course.test` в `192.0.2.10`. Соединение с `192.0.2.10:8080` трижды завершилось по тайм-ауту через 5 секунд. С другого узла того же окружения соединение устанавливается. HTTP-ответ не получен. Приложены точная команда и временные метки.

Во втором варианте уже есть:

- контекст;
- точное время;
- имя, адрес и порт;
- подтверждённый этап отказа;
- сравнительное наблюдение;
- граница того, что ещё не доказано.

Такой запрос помогает SRE или сетевому инженеру начать с проверки, а не с восстановления исходных фактов.

8. Самопроверка

Попробуйте закончить две фразы:

1. «До отправки HTTP-запроса приложение должно…»
2. «Полученный HTTP status code доказывает, что…»

Ожидаемый смысл ответов:

1. Получить или выбрать адрес, установить подходящее transport connection, а для HTTPS также завершить TLS handshake.
2. Некоторый HTTP endpoint сформировал ответ, а клиент получил и смог разобрать его status line; это не гарантирует успех бизнес-операции.

Ещё одна проверка: для фразы «сайт не работает» назовите четыре разных наблюдаемых результата и по одному следующему шагу для каждого. Если ответы начинаются с конкретного этапа и факта, рабочая модель сформирована.

Источники и дальнейшее чтение

- RFC 3986 — Uniform Resource Identifier: Generic Syntax
- RFC 9110 — HTTP Semantics
- RFC 9112 — HTTP/1.1
- RFC 9499 — DNS Terminology
- RFC 1034 — Domain Names: Concepts and Facilities
- RFC 1035 — Domain Names: Implementation and Specification
- RFC 7766 — DNS Transport over TCP
- RFC 2606 — Reserved Top Level DNS Names
- RFC 5737 — IPv4 Address Blocks Reserved for Documentation
- RFC 3849 — IPv6 Address Prefix Reserved for Documentation
- RFC 1122 — Requirements for Internet Hosts
- RFC 8729 — The RFC Series and RFC Editor
- RFC 8700 — Fifty Years of RFCs
- RFC Editor — What Is an RFC?
- RFC 5321 — Simple Mail Transfer Protocol
- RFC 6409 — Message Submission for Mail
- RFC 9051 — Internet Message Access Protocol
- RFC 1939 — Post Office Protocol, Version 3